

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



MÔN HỌC: NHẬP MÔN TRÍ TUỆ NHÂN TẠO

BÁO CÁO BÀI TẬP LỚN

**GIẢI QUYẾT BÀI TOÁN XẾP LỊCH HỌC
BẰNG THUẬT TOÁN DI TRUYỀN (GA)**

GIẢNG VIÊN: TS. NGUYỄN QUỐC MINH

NHÓM THỰC HIỆN: NHÓM 02 - L01

SINH VIÊN:

1. Nguyễn Đức Thịnh - 2313284
2. Lâm Hoàng Vũ - 2313953
3. Lê Thành Bách - 2210182
4. Trần Lê Gia Thoại - 2313323
5. Trần Văn Mạnh - 2212003
6. Trần Trung Hiếu - 2211015
7. Nguyễn Hoàng Danh - 2210456

TP. HỒ CHÍ MINH, THÁNG 05/2026

Tóm tắt (Abstract)

Bài toán lập lịch lớp học (Class Scheduling) là một trong những bài toán tối ưu hóa tổ hợp kinh điển thuộc lớp NP-hard, đòi hỏi việc phân bổ đồng thời nhiều tài nguyên - bao gồm phòng học, khung giờ và giảng viên - cho các lớp học phần sao cho thỏa mãn một tập hợp các ràng buộc phức tạp. Báo cáo này trình bày việc áp dụng Giải thuật Di truyền (Genetic Algorithm - GA) để giải quyết bài toán lập lịch với quy mô 15 môn học, 10 nhóm lớp, 10 giảng viên, 5 ngày học trong tuần, 6 khung giờ mỗi ngày và 30 phòng học thuộc hai loại kích thước khác nhau.

Trong quá trình triển khai, mỗi cá thể trong quần thể được biểu diễn dưới dạng một lịch trình hoàn chỉnh, trong đó mỗi lớp học phần được mã hóa bởi một gene gồm năm thành phần: cặp ngày học, khung giờ buổi thứ nhất, phòng học buổi thứ nhất, khung giờ buổi thứ hai và phòng học buổi thứ hai. Hàm fitness được thiết kế theo công thức $f = 1/(1 + C)$ với C là tổng số xung đột, bao gồm: xung đột về kích thước phòng (phòng nhỏ không đủ sức chứa lớp có đăng ký lớn), xung đột phòng học (hai lớp cùng sử dụng một phòng tại cùng thời điểm), và xung đột giảng viên (một giảng viên bị phân công dạy hai lớp trùng giờ). Quần thể gồm 100 cá thể được tiến hóa qua tối đa 500 thế hệ với cơ chế chọn lọc giải đấu kích thước 3 (tournament selection), lai ghép đồng nhất (uniform crossover) theo từng gene, đột biến ngẫu nhiên với tỷ lệ 10% trên mỗi gene, và bảo toàn cá thể tốt nhất qua cơ chế elitism.

Kết quả thực nghiệm cho thấy giải thuật có khả năng hội tụ nhanh chóng, giảm đáng kể số lượng xung đột qua các thế hệ và trong nhiều lần chạy đạt được lịch trình không có xung đột (**fitness** = 1.0). Quá trình hội tụ được trực quan hóa thông qua biểu đồ số xung đột theo thế hệ, đồng thời được xuất dưới dạng ảnh GIF động để minh họa tiến trình tối ưu hóa. Kết quả khẳng định tính hiệu quả của GA trong việc giải quyết các bài toán lập lịch đa ràng buộc với không gian tìm kiếm lớn.

Mục lục

Tóm tắt (Abstract)	1
Danh sách thành viên & Phân công	4
1 Giới thiệu vấn đề	5
2 Động lực thực hiện	5
2.1 Nhu cầu thực tiễn	5
2.2 Độ phức tạp bài toán	5
2.3 Ưu điểm của Giải thuật Di truyền	6
2.4 Ý nghĩa thực tiễn	6
3 Nghiên cứu liên quan	6
3.1 Bài toán lập lịch	6
3.2 Các phương pháp giải hiện tại	6
3.3 Các nghiên cứu về GA trong lập lịch	7
4 Cơ sở lý thuyết	7
4.1 Giới thiệu Genetic Algorithm	7
4.2 Các thành phần chính của GA	8
4.2.1 Biểu diễn nghiệm	8
4.2.2 Hàm thích nghi	8
4.2.3 Các toán tử di truyền	8
4.3 Chu kỳ thuật toán GA	8
4.4 Hội tụ	8
4.5 Sự phù hợp với bài toán lập lịch	9
5 Thiết kế và hiện thực bài toán	9
5.1 Khởi tạo thông số hệ thống	9
5.2 Biểu diễn cấu trúc thuật toán di truyền	9
6 Hàm Fitness & Xử lý các ràng buộc	10
6.1 Định nghĩa các biến và ký hiệu	10
6.2 Xử lý các ràng buộc (Constraints Handling)	10
6.3 Hàm mục tiêu / Hàm Thích nghi (Fitness Function)	11
7 Phân tích các toán tử di truyền và vòng đời quần thể	11
7.1 Toán tử lựa chọn (Selection)	11
7.2 Toán tử lai ghép (Crossover)	12
7.3 Toán tử đột biến (Mutation)	12
7.4 Quản lý vòng đời quần thể (Population Lifecycle Management)	13
7.5 Đánh giá tổng hợp và hướng cải tiến	13
8 Phân tích thực nghiệm và đánh giá hiệu năng	14
8.1 Thiết lập thực nghiệm	14
8.2 Kết quả với cấu hình mặc định	14
8.2.1 Lịch học tốt nhất tìm được	14
8.2.2 Thống kê qua 20 lần chạy độc lập	15
8.2.3 Biểu đồ tiến trình hội tụ	15
8.3 Khảo sát ảnh hưởng của tham số	16
8.3.1 Ảnh hưởng kích thước quần thể	16
8.3.2 Ảnh hưởng tỷ lệ đột biến	17
8.4 Đánh giá hiệu năng	18
8.4.1 Tốc độ hội tụ	18
8.4.2 Chất lượng nghiệm	18
8.4.3 Tính ổn định	18
8.4.4 Thời gian thực thi	19
8.5 Kết luận mục 6	19

9	Tổng kết: Phân tích Thách thức và Hạn chế	19
9.1	Thách thức	19
9.2	Hạn chế của mô hình	19
10	Kết luận và Hướng phát triển	20
10.1	Kết luận	20
10.2	Hướng phát triển	20
	Tài liệu tham khảo	22

Danh sách thành viên và Phân công công việc

STT	Họ và Tên	MSSV	Nhiệm vụ	% HT
1	Nguyễn Đức Thịnh	2313284	Quản lý dự án, Giới thiệu & Động lực	100%
2	Lâm Hoàng Vũ	2313953	Nghiên cứu liên quan & Cơ sở lý thuyết GA	100%
3	Lê Thành Bách	2210182	Khởi tạo dữ liệu & Biểu diễn cấu trúc thuật toán di truyền	100%
4	Trần Lê Gia Thoại	2313323	Hàm Fitness & Xử lý các ràng buộc	100%
5	Trần Văn Mạnh	2212003	Toán tử Di truyền (Crossover/Mutation) & Quản lý quần thể	100%
6	Trần Trung Hiếu	2211015	Phân tích thực nghiệm & Đánh giá hiệu năng	100%
7	Nguyễn Hoàng Danh	2210456	Thách thức, Hạn chế, Kết luận & References	100%

Bảng 1: Bảng phân công nhiệm vụ nhóm

1 Giới thiệu vấn đề

Bài toán lập lịch lớp học (*Class Scheduling Problem*) tại các trường đại học là một bài toán quản lý tài nguyên mang tính thực tiễn cao, trong đó cần phân bổ đồng thời nhiều loại tài nguyên hữu hạn cho các lớp học phần. Cụ thể, mỗi lớp học phần cần được gán một phòng học phù hợp, một khung giờ cụ thể trong tuần, và một giảng viên phụ trách, đồng thời phải đảm bảo không xảy ra xung đột giữa các lớp khác nhau. Bài toán này thuộc lớp NP-hard do số lượng tổ hợp khả dĩ tăng theo cấp số nhân khi quy mô đầu vào mở rộng, khiến cho các phương pháp tìm kiếm vét cạn trở nên không khả thi ngay cả với quy mô trung bình.

Trong bối cảnh bài tập này, bài toán được mô hình hóa với 15 môn học (`num_courses`) và 15 tên môn (`course_name`) từ 1 đến 15, 10 nhóm lớp học phần (`num_sections`), 10 giảng viên (`num_professors`), 5 ngày học trong tuần (`num_days`) từ thứ Hai đến thứ Sáu, 6 khung giờ mỗi ngày, và 30 phòng học được phân thành hai loại: phòng nhỏ (`room_size = 0`) và phòng lớn (`room_size = 1`). Mỗi lớp học phần cũng có thuộc tính kích thước đăng ký (`class_registration_size`) nhận giá trị 0 hoặc 1, trong đó phòng lớn có thể chứa cả lớp nhỏ lẫn lớp lớn, nhưng phòng nhỏ chỉ phù hợp cho lớp có kích thước đăng ký nhỏ. Một môn học có thể xuất hiện ở nhiều nhóm lớp khác nhau; chẳng hạn, Môn 1 có thể được xếp lịch vào thứ Hai - thứ Tư cho nhóm 01 và đồng thời vào thứ Ba - thứ Năm cho nhóm 02.

Các ràng buộc chính của bài toán bao gồm năm nhóm.

- Thứ nhất, mỗi lớp học phần phải có đúng hai buổi học trong tuần, và hai buổi này phải nằm trên hai ngày không liên tiếp nhau. Ví dụ, cặp thứ Hai - thứ Tư là hợp lệ nhưng thứ Hai - thứ Ba thì không.
- Thứ hai, kích thước phòng học phải tương thích với kích thước đăng ký của lớp: phòng nhỏ chỉ dành cho lớp nhỏ, trong khi phòng lớn có thể phục vụ cả hai loại.
- Thứ ba, một giảng viên không được dạy hai lớp trùng giờ và mỗi giảng viên chỉ được phân công tối đa 3 lớp.
- Thứ tư, các lớp chỉ được xếp vào phòng học khả dụng.
- Thứ năm, một lớp học phần không thể xuất hiện ở 2 phòng khác nhau trong cùng một lúc.
- Thứ sáu, một phòng không được gán cho hai lớp khác nhau tại cùng một thời điểm.

Giải thuật Di truyền (Genetic Algorithm) được lựa chọn làm phương pháp giải quyết do khả năng tìm kiếm hiệu quả trong không gian tổ hợp lớn thông qua cơ chế mô phỏng quá trình tiến hóa tự nhiên. GA hoạt động trên một quần thể các lời giải ứng viên, áp dụng các toán tử chọn lọc, lai ghép và đột biến để dần dần cải thiện chất lượng lời giải qua nhiều thế hệ. Khác với các phương pháp tìm kiếm cục bộ đơn lẻ, GA duy trì sự đa dạng trong quần thể giúp tránh bị kẹt tại các cực trị địa phương, đồng thời vẫn có khả năng hội tụ đến lời giải tối ưu hoặc gần tối ưu. Trong triển khai cụ thể, mỗi lớp học phần chỉ được xếp vào các cặp ngày không liên tiếp đã được xác định trước - gồm sáu cặp hợp lệ: (Thứ 2, Thứ 4), (Thứ 2, Thứ 5), (Thứ 2, Thứ 6), (Thứ 3, Thứ 5), (Thứ 3, Thứ 6), và (Thứ 4, Thứ 6) - nhằm đảm bảo ràng buộc về ngày học không liên tiếp được thỏa mãn ngay từ khâu mã hóa lời giải, giảm thiểu không gian tìm kiếm không hợp lệ.

2 Động lực thực hiện

2.1 Nhu cầu thực tiễn

Động lực chính của nghiên cứu này xuất phát từ nhu cầu thực tiễn trong quản lý đào tạo tại các cơ sở giáo dục đại học. Việc lập thời khóa biểu thủ công cho hàng chục đến hàng trăm lớp học phần, với sự tham gia của nhiều giảng viên, nhiều phòng học có đặc điểm khác nhau, và nhiều khung giờ trong tuần, là một công việc tốn nhiều thời gian, dễ sai sót và khó đảm bảo tính tối ưu. Một lịch trình chứa nhiều xung đột sẽ dẫn đến hàng loạt hệ quả xấu: sinh viên không thể tham gia đầy đủ các buổi học, giảng viên bị xếp trùng giờ, phòng học bị quá tải hoặc không được sử dụng hiệu quả... Do đó, việc tự động hóa quá trình lập lịch bằng các thuật toán tối ưu hóa là một hướng tiếp cận có ý nghĩa cả về mặt học thuật lẫn ứng dụng.

2.2 Độ phức tạp bài toán

Từ góc độ lý thuyết, bài toán lập lịch lớp học là một trường hợp đặc biệt của bài toán thỏa mãn ràng buộc (Constraint Satisfaction Problem - CSP) và cũng có thể được xem là một dạng bài toán tối ưu hóa tổ hợp (Combinatorial Optimization). Không gian tìm kiếm của bài toán tăng theo hàm mũ khi số lượng lớp, phòng, giảng viên và khung giờ gia tăng. Không gian tìm kiếm lên đến hàng tỷ tổ hợp khả dĩ. Các phương pháp duyệt vét cạn hay quy hoạch tuyến tính tuy đảm bảo lời giải tối ưu nhưng không khả thi về mặt thời gian tính toán khi quy mô bài toán tăng, khiến các phương pháp metaheuristic như GA trở thành lựa chọn phù hợp.

2.3 Ưu điểm của Giải thuật Di truyền

Giải thuật Di truyền được chọn nhờ một số ưu điểm nổi bật phù hợp với đặc thù bài toán. Trước hết, GA không yêu cầu hàm mục tiêu phải khả vi hay liên tục, điều này rất quan trọng vì hàm fitness trong bài toán lập lịch có tính chất rời rạc - đếm số xung đột. Thứ hai, cơ chế quần thể (population-based) giúp GA duy trì nhiều lời giải ứng viên đồng thời, từ đó khám phá song song nhiều vùng khác nhau trong không gian tìm kiếm và giảm nguy cơ bị kẹt tại lời giải tối ưu cục bộ. Thứ ba, cấu trúc gene của GA rất linh hoạt, cho phép mã hóa tự nhiên các thành phần của lịch trình (ngày, giờ, phòng) thành từng gene riêng biệt, giúp các toán tử lai ghép và đột biến hoạt động hiệu quả trên từng thành phần. Ngoài ra, việc kết hợp kỹ thuật elitism - bảo toàn cá thể tốt nhất qua các thế hệ - đảm bảo chất lượng lời giải không bao giờ giảm trong quá trình tiến hóa, giúp thuật toán hội tụ ổn định hơn.

2.4 Ý nghĩa thực tiễn

Việc thực hiện dự án này không chỉ giúp giải quyết bài toán cụ thể với 15 môn học và 10 giáo viên, mà còn xây dựng một khung giải thuật (framework) có khả năng mở rộng. Hệ thống này có thể ứng dụng để tối ưu hóa việc sử dụng cơ sở vật chất (phòng học lớn/nhỏ), đảm bảo sức khỏe nghề nghiệp cho giảng viên và nâng cao chất lượng học tập cho sinh viên thông qua việc phân bổ lịch học khoa học.

Định dạng báo cáo

Dự án được triển khai bằng ngôn ngữ Python, viết trong file Jupyter Notebook `class_scheduling.ipynb`. Chương trình sử dụng các thư viện `random`, `numpy`, `seaborn` và `matplotlib`. Kết quả xuất ra bao gồm lịch trình tốt nhất tìm được (best schedule) cùng số xung đột còn lại, và một file ảnh GIF động `ga_scheduling_progress.gif` minh họa quá trình giảm số xung đột của cá thể tốt nhất qua từng thế hệ. Ngoài ra, dự án còn đi kèm một báo cáo PDF trình bày chi tiết nội dung phân tích, phương pháp và kết quả thực nghiệm.

3 Nghiên cứu liên quan

3.1 Bài toán lập lịch

Bài toán lập lịch lớp học (*Class Scheduling Problem*) là một bài toán tối ưu tổ hợp, trong đó mục tiêu là phân bổ các lớp học vào các khung thời gian và phòng học sao cho thỏa mãn một tập các ràng buộc đã xác định trước. Đây là một bài toán có ý nghĩa thực tiễn cao trong các cơ sở giáo dục, vì chất lượng thời khóa biểu ảnh hưởng trực tiếp đến giảng viên, sinh viên và hiệu quả sử dụng tài nguyên.

Các ràng buộc trong bài toán thường được chia thành hai nhóm chính:

- **Hard constraints (ràng buộc cứng):** là các điều kiện bắt buộc phải được thỏa mãn trong mọi nghiệm hợp lệ, như không trùng lịch giữa các lớp do cùng một giảng viên phụ trách, không trùng phòng học tại cùng một thời điểm, và số lượng sinh viên không vượt quá sức chứa của phòng.
- **Soft constraints (ràng buộc mềm):** là các điều kiện nên được ưu tiên đáp ứng để nâng cao chất lượng thời khóa biểu, chẳng hạn bảo đảm phân bổ lịch học hợp lý, hạn chế các buổi học liên tục quá nhiều tiết và ưu tiên các khung giờ thuận lợi.

Do số lượng phương án có thể rất lớn khi quy mô dữ liệu tăng lên, bài toán lập lịch được xem là bài toán *NP-hard*. Vì vậy, việc tìm được nghiệm tối ưu tuyệt đối trong thời gian hợp lý là rất khó, đặc biệt với các hệ thống có nhiều lớp học, giảng viên và phòng học. Trong thực tế, các nghiên cứu thường hướng tới việc tìm nghiệm tốt hoặc gần tối ưu trong thời gian chấp nhận được.

3.2 Các phương pháp giải hiện tại

Trong nhiều năm qua, bài toán lập lịch đã được tiếp cận bằng nhiều phương pháp khác nhau, từ các kỹ thuật tối ưu chính xác đến các phương pháp heuristic và metaheuristic. Bảng 2 trình bày một số hướng tiếp cận phổ biến cùng ưu điểm và hạn chế của chúng.

Trong số các phương pháp trên, *Greedy Heuristic* là cách tiếp cận trực quan và thường được dùng để xây dựng nghiệm ban đầu. Phương pháp này lựa chọn phương án tốt nhất tại từng bước theo một tiêu chí cục bộ. Ưu điểm của cách làm này là tốc độ nhanh và dễ cài đặt, tuy nhiên nó không đảm bảo tối ưu toàn cục và thường cho kết quả chưa tốt khi bài toán có nhiều ràng buộc đan xen.

Phương pháp	Ưu điểm	Nhược điểm
Brute Force	Đảm bảo tìm được nghiệm tối ưu	Độ phức tạp rất lớn, thường ở mức giai thừa, nên không khả thi với dữ liệu thực tế
Integer Programming	Cho nghiệm chính xác, có tính tối ưu toàn cục	Mô hình hóa phức tạp, khó mở rộng cho bài toán lớn và nhiều ràng buộc
Greedy Heuristic	Nhanh, dễ triển khai và chi phí tính toán thấp	Dễ mắc kẹt ở nghiệm cục bộ, chất lượng nghiệm phụ thuộc vào thứ tự lựa chọn
Simulated Annealing	Có khả năng thoát khỏi nghiệm cục bộ trong một số trường hợp	Phụ thuộc nhiều vào tham số như nhiệt độ ban đầu, tốc độ làm nguội
Genetic Algorithm	Linh hoạt, phù hợp với không gian tìm kiếm lớn và ràng buộc phức tạp	Cần lựa chọn cách mã hóa, hàm fitness và tham số phù hợp; thời gian hội tụ có thể dài
Particle Swarm Optimization	Tìm kiếm hiệu quả trong không gian lớn, ý tưởng đơn giản	Khó xử lý trực tiếp các ràng buộc rời rạc và ràng buộc phức tạp của bài toán lập lịch

Bảng 2: So sánh một số phương pháp giải bài toán lập lịch

3.3 Các nghiên cứu về GA trong lập lịch

Genetic Algorithm (GA) là một trong những metaheuristic được nghiên cứu rộng rãi cho bài toán lập lịch nhờ khả năng tìm kiếm hiệu quả trong không gian nghiệm lớn. Nhiều công trình trước đây đã chứng minh rằng GA có thể xử lý tốt các ràng buộc phức tạp nếu được thiết kế với cách biểu diễn và hàm đánh giá phù hợp.

Nhiều nghiên cứu tiêu biểu đã khẳng định tiềm năng của GA trong lĩnh vực này:

- **Colorni et al. (1990):** Nghiên cứu về *university timetabling*, cho thấy GA có khả năng tạo ra nghiệm khả thi trong bối cảnh có nhiều ràng buộc phức tạp.
- **Burke & Newall (2004):** Đề xuất kết hợp GA với các heuristic chuyên biệt nhằm cải thiện chất lượng nghiệm.
- **Abdullah et al. (2007):** Áp dụng mô hình lai giữa GA và *local search*, giúp tăng tốc độ hội tụ và giảm số lượng vi phạm ràng buộc.
- **Sutar & Bichkar (2012):** Tập trung vào thiết kế cơ chế mã hóa chromosome phù hợp với bài toán lập lịch, giúp cải thiện độ ổn định và giảm nghiệm không hợp lệ.

Tổng hợp từ các công trình này có thể thấy rằng GA không chỉ linh hoạt trong cách triển khai mà còn có khả năng thích nghi tốt khi được kết hợp với heuristic, local search hoặc các cơ chế xử lý ràng buộc chuyên biệt, từ đó trở thành một hướng tiếp cận hiệu quả cho bài toán lập lịch.

4 Cơ sở lý thuyết

4.1 Giới thiệu Genetic Algorithm

Genetic Algorithm (GA) là một thuật toán tối ưu dựa trên cơ chế tiến hóa tự nhiên, mô phỏng quá trình chọn lọc và di truyền trong sinh học. Thay vì tìm kiếm lời giải theo cách tuần tự hoặc vét cạn, GA làm việc trên một tập nhiều nghiệm ứng viên, gọi là quần thể (*population*), và cải thiện dần chất lượng của quần thể qua nhiều thế hệ.

Ba cơ chế cốt lõi của GA bao gồm:

- **Selection:** lựa chọn các cá thể tốt để tham gia sinh sản.
- **Crossover:** lai ghép thông tin từ hai cá thể cha mẹ để tạo ra cá thể mới.
- **Mutation:** tạo ra biến đổi ngẫu nhiên nhằm duy trì tính đa dạng của quần thể.

Nhờ cơ chế này, GA có khả năng khám phá không gian nghiệm rộng và dần tiến tới các lời giải có chất lượng tốt hơn sau mỗi vòng lặp.

4.2 Các thành phần chính của GA

4.2.1 Biểu diễn nghiệm

Trong GA, mỗi nghiệm được mã hóa thành một chromosome. Đối với bài toán lập lịch, một chromosome có thể được hiểu là một thời khóa biểu hoàn chỉnh, còn mỗi gene biểu diễn thông tin gán tương ứng cho một lớp học. Cách biểu diễn này có ảnh hưởng trực tiếp đến hiệu quả của thuật toán, vì nó quyết định mức độ dễ dàng khi thực hiện các phép lai ghép, đột biến và đánh giá ràng buộc.

4.2.2 Hàm thích nghi

Fitness function là thành phần dùng để đánh giá chất lượng của mỗi nghiệm. Trong bài toán lập lịch, hàm này thường được xây dựng theo hướng phạt mạnh các vi phạm ràng buộc cứng và phạt nhẹ hơn đối với ràng buộc mềm. Một biểu thức tổng quát có thể được mô tả như sau:

$$Fitness = w_1 \times H + w_2 \times S$$

trong đó H là số lượng vi phạm ràng buộc cứng, S là số lượng vi phạm ràng buộc mềm, còn w_1 và w_2 là các trọng số tương ứng với mức độ ưu tiên. Mục tiêu của thuật toán là tối thiểu hóa giá trị fitness, tức là tìm ra thời khóa biểu có ít vi phạm nhất.

4.2.3 Các toán tử di truyền

Selection (chọn lọc): là bước chọn ra các cá thể tốt trong quần thể để tạo thế hệ mới. Một số kỹ thuật phổ biến là *Roulette Wheel Selection* và *Tournament Selection*. Mục tiêu của bước này là ưu tiên những nghiệm có chất lượng cao hơn nhưng vẫn giữ một phần tính đa dạng cho quần thể.

Crossover (lai ghép): là quá trình kết hợp thông tin của hai chromosome cha mẹ để tạo ra offspring. Các phương pháp phổ biến gồm *single-point crossover* và *two-point crossover*.

Mutation (đột biến): là bước thay đổi ngẫu nhiên một phần chromosome. Toán tử này đóng vai trò quan trọng trong việc tránh hiện tượng mắc kẹt tại nghiệm cục bộ và duy trì khả năng khám phá của thuật toán.

4.3 Chu kỳ thuật toán GA

Quy trình hoạt động cơ bản của Genetic Algorithm có thể được mô tả bằng giả mã sau:

```
1 Initialize population P
2 Evaluate fitness of P
3
4 while (not termination condition):
5     Select parents from P
6     Apply crossover -> offspring
7     Apply mutation
8     Evaluate offspring
9     Select next generation
10
11 Return best solution
```

Listing 1: Chu kỳ hoạt động của Genetic Algorithm

Quy trình trên bắt đầu bằng việc khởi tạo ngẫu nhiên một quần thể nghiệm ban đầu. Sau đó, thuật toán liên tục thực hiện các bước chọn lọc, lai ghép, đột biến và đánh giá để tạo ra thế hệ mới. Qua nhiều vòng lặp, chất lượng nghiệm tốt nhất trong quần thể sẽ dần được cải thiện.

4.4 Hội tụ

Trong GA, hiện tượng hội tụ xảy ra khi chất lượng nghiệm không còn cải thiện đáng kể sau nhiều thế hệ, hoặc khi thuật toán đạt đến một ngưỡng dừng đã xác định trước. Một số yếu tố ảnh hưởng mạnh đến tốc độ và chất lượng hội tụ gồm:

- kích thước quần thể (*population size*),
- xác suất đột biến (*mutation rate*),
- áp lực chọn lọc (*selection pressure*).

Nếu áp lực chọn lọc quá cao hoặc quần thể thiếu đa dạng, thuật toán có thể rơi vào hiện tượng *premature convergence*, tức là hội tụ sớm tại một nghiệm cục bộ thay vì tiếp tục khám phá các vùng nghiệm tốt hơn.

4.5 Sự phù hợp với bài toán lập lịch

GA được xem là phù hợp với bài toán lập lịch vì nhiều lý do. Thứ nhất, thuật toán không yêu cầu thông tin gradient hay một mô hình toán học quá chặt chẽ như các phương pháp tối ưu cổ điển. Thứ hai, GA có khả năng làm việc tốt với không gian nghiệm lớn, rời rạc và chứa nhiều ràng buộc phức tạp. Thứ ba, cấu trúc của GA linh hoạt, cho phép dễ dàng tùy chỉnh cách mã hóa nghiệm, hàm fitness và các toán tử di truyền theo đặc thù của từng bài toán.

So với các phương pháp khác, GA thường cho chất lượng nghiệm tốt hơn so với các thuật toán Greedy thuần túy, dễ triển khai hơn so với mô hình Integer Programming trong nhiều trường hợp thực tế, và có thể ổn định hơn Simulated Annealing nếu được lựa chọn tham số phù hợp. Chính vì vậy, GA là một hướng tiếp cận tiềm năng và thực tiễn cho bài toán xây dựng thời khóa biểu.

5 Thiết kế và hiện thực bài toán

5.1 Khởi tạo thông số hệ thống

Trong bài toán lập lịch lớp học, dữ liệu đầu vào được khởi tạo dựa trên các thành phần chính của hệ thống đào tạo, bao gồm danh sách môn học, lớp học phần, giảng viên, ngày học, khung giờ và phòng học. Các thông số này đóng vai trò là nền tảng để thuật toán di truyền xây dựng và tối ưu các phương án thời khóa biểu.

Cụ thể, hệ thống xét 15 môn học, 10 giảng viên, 5 ngày học trong tuần, 6 khung giờ mỗi ngày và 30 phòng học. Mỗi phòng học được gán ngẫu nhiên một loại kích thước, trong đó giá trị 0 biểu diễn phòng nhỏ và giá trị 1 biểu diễn phòng lớn. Tương tự, mỗi lớp học phần cũng có quy mô đăng ký thuộc một trong hai loại: lớp nhỏ (0) hoặc lớp lớn (1). Ràng buộc về sức chứa được mô hình hóa theo nguyên tắc: phòng lớn có thể phục vụ cả lớp nhỏ và lớp lớn, trong khi phòng nhỏ chỉ phù hợp với lớp nhỏ.

Trong phần hiện thực, phòng học được biểu diễn bởi lớp `Room`, bao gồm mã phòng và kích thước phòng. Mỗi lớp học phần được biểu diễn bởi lớp `ClassSection`, bao gồm mã lớp học phần, mã môn học, mã giảng viên phụ trách và quy mô đăng ký của lớp. Việc khởi tạo dữ liệu được thực hiện ngẫu nhiên nhằm tạo ra các trường hợp thử nghiệm đa dạng cho thuật toán di truyền.

Một điểm quan trọng trong quá trình khởi tạo là ràng buộc mỗi giảng viên không phụ trách quá 3 lớp học phần. Ràng buộc này được kiểm soát thông qua biến đếm số lớp của từng giảng viên. Khi tạo một lớp học phần mới, thuật toán chỉ chọn ngẫu nhiên trong nhóm giảng viên hiện vẫn còn số lớp phụ trách nhỏ hơn 3. Cách làm này giúp đảm bảo ràng buộc về tải giảng dạy của giảng viên được xử lý ngay từ giai đoạn sinh dữ liệu ban đầu, thay vì chỉ xử lý trong quá trình đánh giá lịch.

Đối với ràng buộc mỗi lớp có hai buổi học trong tuần, chương trình không chọn hai ngày học hoàn toàn ngẫu nhiên mà định nghĩa trước tập các cặp ngày hợp lệ:

$$(0, 2), (0, 3), (0, 4), (1, 3), (1, 4), (2, 4).$$

Trong đó, ánh xạ các ngày 0, 1, 2, 3, 4 lần lượt tương ứng với Thứ Hai đến Thứ Sáu, các cặp ngày trên bảo đảm rằng hai buổi học của cùng một lớp không diễn ra trong cùng ngày và cũng không rơi vào hai ngày liên kề. Nhờ đó, ràng buộc về khoảng cách giữa hai buổi học được bảo đảm trực tiếp thông qua miền giá trị của dữ liệu, giúp giảm số lượng lời giải không hợp lệ trong quá trình tiến hóa.

Trong phần hiện thực, số lớp học phần được khởi tạo dựa trực tiếp trên biến `NUM_SECTIONS`. Cụ thể, chương trình tạo ra 10 lớp học phần với mã lớp từ 0 đến 9. Đối với mỗi lớp, một mã môn học được chọn ngẫu nhiên trong khoảng từ 1 đến `NUM_COURSES`. Nhờ đó, nhiều lớp học phần khác nhau có thể thuộc cùng một môn học, phản ánh đúng thực tế rằng một môn học có thể được mở cho nhiều section hoặc nhiều nhóm sinh viên khác nhau. Mỗi lớp học phần sau đó được gán giảng viên phụ trách và quy mô đăng ký lớp thông qua quá trình khởi tạo ngẫu nhiên, tạo ra tập dữ liệu ban đầu để thuật toán di truyền tiến hành tối ưu lịch học.

5.2 Biểu diễn cấu trúc thuật toán di truyền

Trong thuật toán di truyền, mỗi cá thể trong quần thể biểu diễn một phương án thời khóa biểu hoàn chỉnh cho toàn bộ các lớp học phần. Một cá thể được cài đặt bằng lớp `Schedule`, trong đó thuộc tính `genes` lưu thông tin lịch học của từng lớp học phần, còn `fitness` và `conflicts` lần lượt biểu diễn độ thích nghi và số lượng xung đột của lịch.

Mỗi gene tương ứng với lịch học của một lớp học phần và được biểu diễn dưới dạng một vector gồm năm thành phần:

$$[day_pair, slot_1, room_1, slot_2, room_2].$$

Trong đó, `day_pair` là chỉ số của cặp ngày học được chọn từ tập các cặp ngày hợp lệ; `slot1` và `slot2` lần lượt là khung giờ của hai buổi học; `room1` và `room2` là phòng học tương ứng cho từng buổi. Cách biểu diễn này cho

phép một lớp có hai buổi học trong tuần, đồng thời cho phép hai buổi học có thể diễn ra ở hai phòng khác nhau và hai khung giờ khác nhau.

Quần thể ban đầu được sinh ngẫu nhiên với kích thước 100 cá thể. Đối với mỗi cá thể, chương trình lần lượt sinh lịch cho tất cả các lớp học phần bằng cách chọn ngẫu nhiên cặp ngày hợp lệ, khung giờ và phòng học cho từng buổi. Sau khi khởi tạo, cá thể được đánh giá bằng hàm thích nghi.

Hàm đánh giá độ thích nghi dựa trên số lượng xung đột trong lịch. Ba nhóm xung đột chính được xem xét gồm: xung đột về kích thước phòng, xung đột sử dụng phòng và xung đột giảng viên. Thứ nhất, nếu lớp học có quy mô lớn nhưng được gán vào phòng nhỏ thì lịch bị cộng thêm một lỗi. Thứ hai, nếu hai lớp học phần sử dụng cùng một phòng tại cùng ngày và cùng khung giờ thì phát sinh xung đột phòng. Thứ ba, nếu một giảng viên được phân công dạy hai lớp khác nhau tại cùng ngày và cùng khung giờ thì phát sinh xung đột giảng viên.

Giá trị thích nghi của một cá thể được tính theo công thức:

$$fitness = \frac{1}{1 + conflicts}.$$

Công thức này bảo đảm rằng lịch có số xung đột càng ít thì giá trị thích nghi càng cao. Khi số xung đột bằng 0, cá thể đạt độ thích nghi tối đa là 1.0, tương ứng với một lịch hợp lệ theo các ràng buộc đang được kiểm tra.

Quá trình tiến hóa được thực hiện trong tối đa 500 thế hệ. Ở mỗi thế hệ, quần thể được sắp xếp theo độ thích nghi giảm dần để xác định cá thể tốt nhất. Thuật toán sử dụng chiến lược elitism bằng cách giữ lại trực tiếp cá thể tốt nhất sang thế hệ tiếp theo. Điều này giúp tránh làm mất lời giải tốt nhất đã tìm được trong quá trình lai ghép và đột biến.

Đối với bước chọn lọc, chương trình sử dụng tournament selection với kích thước mẫu là 3. Cụ thể, ba cá thể được chọn ngẫu nhiên từ quần thể, sau đó cá thể có độ thích nghi cao nhất trong nhóm này được chọn làm cha hoặc mẹ. Cách chọn lọc này cân bằng giữa việc ưu tiên các cá thể tốt và duy trì tính đa dạng trong quần thể.

Toán tử lai ghép được thực hiện ở mức gene của từng lớp học phần. Với mỗi lớp, lịch của lớp đó trong cá thể con được sao chép từ một trong hai cá thể cha mẹ với xác suất ngẫu nhiên. Nhờ đó, cá thể con có thể kế thừa các phần lịch tốt từ cả hai cá thể cha mẹ.

Sau lai ghép, toán tử đột biến được áp dụng với xác suất mặc định là 0.1. Khi một gene được chọn để đột biến, một trong năm thành phần của gene sẽ được thay đổi ngẫu nhiên, có thể là cặp ngày học, khung giờ hoặc phòng học. Đột biến giúp thuật toán khám phá thêm các vùng lời giải mới và hạn chế hiện tượng hội tụ sớm.

Trong quá trình chạy, chương trình lưu lại lịch sử giá trị fitness tốt nhất và số xung đột thấp nhất qua từng thế hệ. Các giá trị này được sử dụng để trực quan hóa quá trình tối ưu của thuật toán di truyền, ví dụ thông qua biểu đồ hoặc ảnh động thể hiện số xung đột giảm dần theo thời gian. Cá thể tốt nhất sau khi thuật toán kết thúc được lưu lại trong biến `best_schedule` và được xem là phương án lịch học cuối cùng.

6 Hàm Fitness & Xử lý các ràng buộc

6.1 Định nghĩa các biến và ký hiệu

Gọi tập hợp các lớp học (Class Section) là C . Mỗi lớp học $c \in C$ sẽ được gán một bộ gen đại diện cho lịch học trong tuần bao gồm 2 buổi:

- d_1, d_2 : Ngày học của buổi 1 và buổi 2.
- s_1, s_2 : Ca học (timeslot) của buổi 1 và buổi 2.
- r_1, r_2 : Phòng học của buổi 1 và buổi 2.
- p_c : Giảng viên phụ trách lớp c .
- S_c : Sức chứa yêu cầu của lớp c .
- S_r : Sức chứa thực tế của phòng học r .

Tổng số lỗi (Conflicts) của một nhiễm sắc thể (một cách xếp lịch) được ký hiệu là K . Hệ thống xử lý 3 ràng buộc chính để tính K .

6.2 Xử lý các ràng buộc (Constraints Handling)

Ràng buộc 1: Sức chứa của phòng học (Room Capacity Constraint)

Phòng học được xếp phải có sức chứa lớn hơn hoặc bằng sĩ số/sức chứa yêu cầu của lớp học đó. Hàm phạt lỗi sức chứa K_{cap} được tính bằng tổng số lần vi phạm trong cả 2 buổi học của tất cả các lớp:

$$K_{cap} = \sum_{c \in C} (I(S_{r_1} < S_c) + I(S_{r_2} < S_c))$$

(Trong đó $I(x)$ là hàm chỉ báo, nhận giá trị 1 nếu điều kiện x đúng, và 0 nếu sai).

Ràng buộc 2: Trùng lặp phòng học (Room Overlap Constraint)

Tại bất kỳ một ngày d và một ca học s cụ thể, một phòng học r chỉ được phép tổ chức tối đa 1 lớp học. Gọi $N(d, s, r)$ là tổng số lớp được xếp vào phòng r tại ngày d , ca s . Số lỗi trùng phòng K_{room} là:

$$K_{\text{room}} = \sum_d \sum_s \sum_r \max(0, N(d, s, r) - 1)$$

Ràng buộc 3: Trùng lặp giảng viên (Professor Overlap Constraint)

Tương tự như phòng học, tại bất kỳ một ngày d và ca học s cụ thể, một giảng viên p chỉ có thể dạy tối đa 1 lớp học. Gọi $N(d, s, p)$ là tổng số lớp mà giảng viên p phải dạy tại ngày d , ca s . Số lỗi trùng lịch giảng viên K_{prof} là:

$$K_{\text{prof}} = \sum_d \sum_s \sum_p \max(0, N(d, s, p) - 1)$$

6.3 Hàm mục tiêu / Hàm Thích nghi (Fitness Function)

Tổng số lỗi của một lịch học là tổng vi phạm của cả 3 ràng buộc trên:

$$\text{Conflicts} = K_{\text{cap}} + K_{\text{room}} + K_{\text{prof}}$$

Mục tiêu của Thuật toán Di truyền (GA) là tìm ra lịch học sao cho $\text{Conflicts} \rightarrow 0$. Do thuật toán GA thường thiên về việc tìm giá trị lớn nhất (Maximize) cho hàm Fitness, nên hàm Fitness F được thiết kế nghịch đảo với tổng số lỗi:

$$F = \frac{1}{1 + \text{Conflicts}}$$

Đánh giá hàm Fitness:

- Khi $\text{Conflicts} = 0$ (Lịch học hoàn hảo, không vi phạm ràng buộc nào): $F = 1.0$.
- Khi $\text{Conflicts} > 0$: F sẽ tiến dần về 0. Số lỗi càng nhiều, F càng nhỏ.
- Việc cộng thêm 1 ở mẫu số nhằm tránh lỗi chia cho 0 trong trường hợp tìm được lịch học tối ưu hoàn toàn.

7 Phân tích các toán tử di truyền và vòng đời quần thể

Sau khi đã xác định cách biểu diễn nhiễm sắc thể và hàm thích nghi, hiệu quả thực sự của thuật toán di truyền phụ thuộc chủ yếu vào bốn cơ chế vận hành cốt lõi: lựa chọn, lai ghép, đột biến và quản lý vòng đời quần thể. Đây là các thành phần quyết định cách quần thể tiến hóa theo thời gian, mức độ khai thác các nghiệm tốt đang có, cũng như khả năng khám phá thêm những vùng lời giải mới trong không gian tìm kiếm rất lớn của bài toán lập lịch lớp học. Trong phần hiện thực của đề tài, bốn cơ chế này được cài đặt tương đối gọn nhưng thể hiện rõ tư tưởng của một mô hình GA cổ điển theo kiểu sinh thế hệ mới hoàn toàn (generational GA) như cách trình bày trong các tài liệu nền tảng về tính toán tiến hóa [7, 14].

Xét trên ngữ cảnh bài toán, mỗi cá thể là một lịch học hoàn chỉnh cho toàn bộ tập lớp học phần; do đó, một thay đổi nhỏ trong một gene vẫn có thể ảnh hưởng dây chuyền đến số lượng xung đột về phòng hoặc giảng viên. Vì vậy, việc phân tích riêng từng toán tử là cần thiết để lý giải vì sao thuật toán có thể hội tụ, đồng thời cũng giúp chỉ ra các giới hạn của thiết kế hiện tại và các hướng cải tiến khả thi.

7.1 Toán tử lựa chọn (Selection)

Trong chương trình, bước chọn cha mẹ được thực hiện bằng *tournament selection* với kích thước giải đấu bằng 3. Cụ thể, để chọn ra một cá thể cha hoặc mẹ, thuật toán lấy ngẫu nhiên 3 cá thể từ quần thể hiện tại, sau đó chọn cá thể có giá trị fitness lớn nhất trong nhóm này. Quá trình được lặp lại độc lập hai lần để thu được hai cá thể phục vụ cho lai ghép.

Ưu điểm lớn nhất của tournament selection là cách cài đặt đơn giản, không phụ thuộc vào giá trị tuyệt đối của fitness mà chỉ dựa trên quan hệ thứ hạng giữa các cá thể. Điều này rất phù hợp với bài toán hiện tại vì hàm fitness có dạng

$$F = \frac{1}{1 + \text{Conflicts}},$$

trong đó chênh lệch giữa các cá thể ở giai đoạn đầu và giai đoạn cuối của quá trình tiến hóa có thể rất khác nhau. Nếu sử dụng các phương pháp như *roulette wheel selection*, sự thay đổi thang đo fitness theo từng giai đoạn có thể tạo ra áp lực chọn lọc không ổn định; ngược lại, tournament selection duy trì được tính nhất quán hơn. Về mặt lý thuyết, dạng chọn lọc này cũng đã được phân tích khá sâu về áp lực chọn lọc theo kích thước giải đấu [9].

Với kích thước giải đấu $k = 3$ và quần thể 100 cá thể, áp lực chọn lọc thu được ở mức vừa phải: các cá thể tốt có xác suất được chọn cao hơn rõ rệt, nhưng các cá thể trung bình vẫn còn cơ hội tham gia sinh sản. Đây là điểm cân bằng quan trọng. Nếu k quá nhỏ, quá trình tiến hóa gần như ngẫu nhiên và hội tụ chậm; nếu k quá lớn, quần thể dễ bị chi phối bởi một vài cá thể nổi trội và nhanh chóng mất đa dạng. Đối với bài toán lập lịch với nhiều cực trị địa phương, việc tránh hội tụ sớm là đặc biệt quan trọng [7, 9].

Một điểm đáng chú ý là hai lần chọn p_1 và p_2 được thực hiện độc lập, nên về mặt kỹ thuật cùng một cá thể có thể được chọn làm cả cha lẫn mẹ. Điều này không sai về thuật toán, nhưng nếu xảy ra thường xuyên ở giai đoạn cuối, mức đa dạng của thế hệ con có thể suy giảm. Dù vậy, với tournament size bằng 3 và có thêm đột biến ở bước sau, hiện tượng này vẫn được kiểm soát ở mức chấp nhận được.

Tóm lại, lựa chọn bằng tournament selection là một quyết định hợp lý cho bài toán này vì vừa dễ triển khai, vừa tạo đủ áp lực tiến hóa để đẩy quần thể tiến dần về các lịch có ít xung đột hơn. Toán tử này đóng vai trò định hướng quá trình tìm kiếm bằng cách ưu tiên truyền lại vật liệu di truyền từ các cá thể có lịch học khả thi hơn.

7.2 Toán tử lai ghép (Crossover)

Sau khi chọn được hai cá thể cha mẹ, chương trình sử dụng *uniform crossover* ở mức gene của từng lớp học phần. Với mỗi lớp trong tập `classes`, lịch của lớp đó trong cá thể con được sao chép toàn bộ từ cha hoặc từ mẹ với xác suất xấp xỉ 0.5. Nói cách khác, đơn vị trao đổi của phép lai không phải là từng thành phần riêng lẻ như ngày, slot hay phòng, mà là toàn bộ vector năm phần tử:

$$[day_pair, slot_1, room_1, slot_2, room_2].$$

Cách thiết kế này có một ưu điểm quan trọng: nó bảo toàn tính nhất quán nội bộ của lịch cho từng lớp học phần. Một lớp đã được cha hoặc mẹ gán đầy đủ cặp ngày học, khung giờ và phòng học cho cả hai buổi, nên khi sao chép nguyên khối sang con, cấu trúc nội tại của lớp đó không bị phá vỡ. Điều này phù hợp hơn so với việc trộn từng thành phần nhỏ trong gene, bởi nếu lấy *day_pair* từ cha nhưng *slot* hoặc *room* từ mẹ thì rất dễ tạo ra những tổ hợp ngẫu nhiên kém ý nghĩa hơn.

So với *one-point crossover* hoặc *two-point crossover*, uniform crossover cũng tránh được sự phụ thuộc vào thứ tự sắp xếp các lớp trong nhiễm sắc thể. Trong bài toán này, thứ tự của các `class_id` chủ yếu mang tính kỹ thuật chứ không phản ánh mối liên hệ bản chất giữa các lớp học. Vì vậy, việc cắt nhiễm sắc thể theo một điểm hay hai điểm cố định có thể tạo ra thiên lệch vị trí không cần thiết. Uniform crossover linh hoạt hơn vì xem mỗi lớp là một đơn vị độc lập để trao đổi; đây cũng là tinh thần của phép lai ghép uniform được Syswerda đề xuất [10].

Tuy nhiên, thiết kế hiện tại vẫn tồn tại hai hạn chế. Thứ nhất, chương trình không sử dụng *crossover rate*; tức là mọi cặp cha mẹ được chọn đều luôn sinh con thông qua lai ghép, thay vì đôi khi giữ nguyên một cá thể tốt để truyền thẳng sang thế hệ mới. Thứ hai, uniform crossover có xu hướng trộn mạnh vật liệu di truyền, nên tuy tăng khả năng khám phá, nó cũng có thể phá vỡ các tổ hợp lớp học đang phối hợp tốt với nhau ở quy mô toàn cục. Ví dụ, hai lớp khác nhau có thể đang được sắp xếp tránh trùng giảng viên và trùng phòng nhờ một cấu hình thời gian tương thích; việc lấy lớp này từ cha và lớp kia từ mẹ có thể làm mất sự hài hòa đó. Nhận xét này phù hợp với các phân tích cổ điển về *building blocks* và nguy cơ phá vỡ lược đồ tốt nếu lai ghép quá mạnh [6, 7].

Dù vậy, trong bối cảnh đề tài này, lựa chọn uniform crossover ở mức từng lớp học phần vẫn là phương án hợp lý. Nó cho phép cá thể con kế thừa các "mảnh lịch" tốt từ cả hai cha mẹ, đồng thời giữ được cấu trúc gene rõ ràng và dễ cài đặt.

7.3 Toán tử đột biến (Mutation)

Sau bước lai ghép, cá thể con tiếp tục đi qua toán tử đột biến với xác suất mặc định là 0.1 trên mỗi lớp học phần. Cơ chế đột biến được cài đặt theo kiểu *random resetting*: nếu một lớp được chọn để đột biến, chương trình chọn ngẫu nhiên một trong năm thành phần của gene rồi gán lại thành phần đó bằng một giá trị ngẫu nhiên hợp lệ trong miền tương ứng.

Đây là kiểu đột biến phù hợp với bài toán có biến rời rạc. Các gene trong mô hình không phải là biến liên tục mà là chỉ số của cặp ngày, khung giờ và phòng học. Vì vậy, những phép đột biến kiểu cộng trừ vi sai nhỏ không thật sự tự nhiên bằng việc đặt lại trực tiếp một giá trị mới. Đồng thời, do mỗi lần đột biến chỉ tác động lên một trong năm thành phần của một lớp, mức độ thay đổi là tương đối cục bộ, không phá hủy toàn bộ nhiễm sắc thể.

Vai trò chính của đột biến là duy trì tính đa dạng của quần thể và giúp thuật toán thoát khỏi các cực trị địa phương. Trong bài toán lập lịch, nhiều lịch học có thể chỉ khác nhau ở một vài quyết định nhỏ về phòng hoặc khung giờ, nhưng chính các thay đổi nhỏ đó lại quyết định việc có phát sinh xung đột hay không. Nhờ đột biến, thuật toán có cơ hội khám phá những vùng lời giải mà chỉ dùng chọn lọc và lai ghép khó tiếp cận.

Với tỉ lệ 0.1, nếu quần thể có khoảng 25 lớp học phần thì trung bình mỗi cá thể con sẽ có khoảng 2 đến 3 lớp bị đột biến sau mỗi lần sinh. Đây là mức đột biến khá mạnh nếu so với các khuyến nghị cổ điển kiểu $1/L$ với L là tổng số gene, nhưng lại có thể chấp nhận được trong bối cảnh bài toán nhiều ràng buộc và không gian tìm kiếm gồ ghề. Nói cách khác, mức đột biến tương đối cao giúp tăng khả năng thăm dò không gian nghiệm, bù lại cho việc cấu trúc lịch học có nhiều điểm mắc kẹt cục bộ. Trong lý thuyết GA, khuyến nghị dùng xác suất đột biến nhỏ đã được thảo luận từ rất sớm bởi De Jong, và về sau được mở rộng trong các nghiên cứu của Mühlenbein và Bäck [8, 11, 12].

Tuy nhiên, mặt trái của lựa chọn này là nguy cơ phá vỡ các cá thể đã tốt. Khi một lịch học đã gần đạt nghiệm khả thi, việc thay đổi ngẫu nhiên một thành phần vẫn có thể làm phát sinh thêm xung đột mới. Chính vì vậy, đột biến mạnh chỉ thực sự hiệu quả khi được kết hợp với cơ chế elitism trong bước quản lý vòng đời quần thể. Có thể xem đây là sự đánh đổi có chủ đích: chấp nhận cho đa số cá thể con biến động mạnh hơn để đổi lấy khả năng khám phá, nhưng vẫn giữ lại ít nhất một cá thể tốt nhất nhằm bảo đảm quá trình tiến hóa không bị "trôi lùi".

7.4 Quản lý vòng đời quần thể (Population Lifecycle Management)

Về tổng thể, chương trình sử dụng mô hình *generational replacement*, tức là ở mỗi thế hệ, gần như toàn bộ quần thể cũ được thay bằng một quần thể mới sinh ra từ quá trình chọn lọc, lai ghép và đột biến. Trước khi sinh thế hệ mới, quần thể hiện tại được sắp xếp giảm dần theo fitness để xác định cá thể tốt nhất. Nếu cá thể đầu tiên đã có số xung đột bằng 0, thuật toán dừng sớm vì đã tìm được lịch hợp lệ hoàn toàn. Nếu chưa đạt nghiệm tối ưu, thuật toán tạo `new_population` và sao chép trực tiếp cá thể tốt nhất của thế hệ hiện tại vào quần thể mới.

Cơ chế trên chính là *1-elitism*. Vai trò của nó rất quan trọng: trong một thuật toán có lai ghép mạnh và đột biến khá cao như mô hình hiện tại, nếu không giữ lại cá thể tốt nhất thì hoàn toàn có thể xảy ra tình huống thế hệ sau kém hơn thế hệ trước. Elitism bảo đảm rằng chất lượng tốt nhất đã đạt được sẽ không bị mất đi qua các thế hệ. Điều này làm cho giá trị fitness cực đại của quần thể có xu hướng không giảm theo thời gian [8, 14].

Sau khi giữ lại cá thể elite, thuật toán tiếp tục sinh các cá thể con cho đến khi đủ kích thước quần thể ban đầu. Như vậy, kích thước quần thể luôn cố định ở mức 100. Việc duy trì quy mô cố định giúp dễ kiểm soát chi phí tính toán và tạo nhịp tiến hóa ổn định qua các thế hệ. Tiêu chí dừng gồm hai điều kiện: hoặc tìm được lịch không còn xung đột, hoặc đạt đến số thế hệ tối đa là 500. Hai điều kiện này tạo ra một cơ chế dừng vừa thực dụng vừa hiệu quả: thuật toán không lãng phí thời gian nếu đã tìm được nghiệm tốt, nhưng cũng không chạy vô hạn nếu chưa hội tụ.

Xét về mặt ưu điểm, mô hình vòng đời này rõ ràng, dễ hiện thực và phù hợp với quy mô dữ liệu của bài tập lớn. Generational GA kết hợp với elitism cho phép quan sát quá trình cải thiện của quần thể theo từng thế hệ một cách trực quan, rất thuận lợi cho việc vẽ biểu đồ hội tụ và phân tích thực nghiệm. Ngoài ra, vì toàn bộ quần thể mới được sinh ra liên tục từ các phép kết hợp khác nhau, thuật toán vẫn giữ được tính khám phá khá tốt.

Tuy nhiên, thiết kế này cũng bộc lộ một số giới hạn. Thứ nhất, việc chỉ giữ lại đúng một cá thể tốt nhất có thể chưa đủ để bảo tồn nhiều cấu trúc lời giải triển vọng khác nhau. Thứ hai, mô hình thay thế toàn phần dễ làm mất đa dạng nhanh nếu quần thể bắt đầu bị chi phối bởi một số ít cá thể mạnh. Thứ ba, các tham số như kích thước giải đấu, tỉ lệ đột biến hay cách sinh con đều là cố định, nên thuật toán chưa thích nghi linh hoạt với từng giai đoạn của quá trình tiến hóa. Chẳng hạn, ở giai đoạn đầu có thể cần khám phá mạnh hơn, còn ở giai đoạn cuối lại cần tinh chỉnh nhẹ để ổn định lời giải. Các hướng điều khiển tham số thích nghi và biến thể thay thế quần thể như *steady-state* hay $(\mu + \lambda)$ đã được bàn luận khá rộng trong tài liệu về evolutionary computing [14, 13].

Nhìn chung, cơ chế quản lý vòng đời quần thể trong đề tài đã hoàn thành tốt vai trò khung điều phối chung cho toàn bộ quá trình tiến hóa. Sự kết hợp giữa sinh thế hệ mới hoàn toàn, elitism, kích thước quần thể cố định và dừng sớm tạo nên một quy trình đơn giản nhưng hiệu quả, đủ mạnh để giải quyết bài toán lập lịch ở quy mô trung bình.

7.5 Đánh giá tổng hợp và hướng cải tiến

Từ bốn thành phần đã phân tích, có thể thấy thiết kế hiện tại của thuật toán di truyền đạt được sự cân bằng tương đối hợp lý giữa khai thác và thăm dò. Tournament selection tạo áp lực chọn lọc ổn định; uniform crossover giúp trộn các mảnh lịch tốt ở mức lớp học phần; random-reset mutation duy trì đa dạng và hỗ trợ thoát cực trị địa phương; còn generational replacement kết hợp 1-elitism bảo đảm quần thể tiến hóa theo hướng không làm mất nghiệm tốt nhất đã tìm được.

Điểm mạnh của cấu hình này nằm ở tính đơn giản, rõ ràng và dễ cài đặt. Mỗi toán tử đều bám sát đặc trưng biểu diễn của bài toán, không đòi hỏi cấu trúc dữ liệu phức tạp và đủ hiệu quả để đưa số lượng xung đột giảm nhanh qua các thế hệ. Đây là một ưu điểm quan trọng trong bối cảnh bài tập lớn, nơi mục tiêu không chỉ là tìm nghiệm tốt mà còn phải trình bày được quy trình hiện thực một cách dễ hiểu và có thể kiểm chứng.

Mặc dù vậy, vẫn còn nhiều hướng cải tiến đáng cân nhắc nếu muốn tăng chất lượng lời giải hoặc cải thiện tốc độ hội tụ. Một khả năng là bổ sung *crossover rate* để không phải mọi lần chọn cha mẹ đều bắt buộc sinh con bằng lai ghép. Một hướng khác là dùng *adaptive mutation rate*, trong đó tỉ lệ đột biến giảm dần theo số thế hệ hoặc thay đổi tùy theo mức độ đa dạng hiện tại của quần thể. Ngoài ra, có thể mở rộng elitism từ 1 cá thể lên một số nhỏ cá thể tốt nhất, hoặc kết hợp thêm *repair operator* để sửa nhanh các xung đột rõ ràng sau khi lai ghép và đột biến. Trong miền bài toán xếp lịch đại học, các hướng kết hợp GA với heuristic sửa lỗi hoặc tìm kiếm cục bộ thường cho kết quả tốt hơn so với chỉ dùng penalty thuần túy [15, 4, 3].

Cuối cùng, nếu xét dưới góc độ nghiên cứu sâu hơn, mô hình hiện tại có thể được phát triển thành các biến thể mạnh hơn như steady-state GA, memetic algorithm hoặc GA kết hợp tìm kiếm cục bộ. Tuy nhiên, ngay trong cấu hình hiện tại, bốn toán tử di truyền và cơ chế quản lý vòng đời quần thể đã tạo thành một hệ thống nhất quán, đủ để chứng minh tính khả thi của GA đối với bài toán xếp lịch lớp học đa ràng buộc.

8 Phân tích thực nghiệm và đánh giá hiệu năng

8.1 Thiết lập thực nghiệm

Để đánh giá thuật toán di truyền (GA) một cách khách quan, nhóm tiến hành thực nghiệm trên cùng một bài toán đã được khởi tạo bằng seed cố định (`seed_setup = 42`), gồm **25 lớp học** và **30 phòng học** (xấp xỉ 50% phòng nhỏ và 50% phòng lớn). Mỗi giáo viên dạy tối đa 3 lớp.

Cấu hình mặc định của GA gồm: quần thể **100 cá thể**, số thế hệ tối đa **500**, lai ghép **đồng nhất theo gene** (uniform per-gene crossover), đột biến **đa điểm** với tỷ lệ **0.10**, chọn lọc **giải đấu** kích thước 3, và cơ chế *elitism* giữ lại cá thể tốt nhất qua mỗi thế hệ. Tiêu chí dừng là đạt 0 xung đột hoặc chạy hết 500 thế hệ.

Mỗi cấu hình được chạy lặp lại với các seed độc lập để khử nhiễu ngẫu nhiên: **20 lần** với cấu hình mặc định và **10 lần** cho mỗi cấu hình khảo sát (tổng cộng 70 lần chạy độc lập). Hàm thích nghi được định nghĩa $fitness = 1/(1 + \text{số xung đột})$ nên giá trị tối đa là 1.0 tương ứng với 0 xung đột.

8.2 Kết quả với cấu hình mặc định

8.2.1 Lịch học tốt nhất tìm được

Trong 20 lần chạy độc lập, mọi lần chạy đều đạt $fitness = 1.0$ (0 xung đột). Lần chạy nhanh nhất hội tụ chỉ sau **16 thế hệ** (seed 1001).

Bảng 3: Bảng phân công lịch

Lớp	Môn	GV	Cặp ngày	Tiết–Phòng buổi 1	Tiết–Phòng buổi 2
S00	C7	P2	T2–T6	Tiết 5 / Phòng 24	Tiết 4 / Phòng 24
S01	C14	P6	T2–T6	Tiết 6 / Phòng 18	Tiết 5 / Phòng 24
S02	C13	P1	T2–T6	Tiết 6 / Phòng 8	Tiết 1 / Phòng 19
S03	C9	P2	T4–T6	Tiết 6 / Phòng 19	Tiết 6 / Phòng 19
S04	C2	P9	T4–T6	Tiết 5 / Phòng 23	Tiết 6 / Phòng 24
S05	C14	P10	T2–T5	Tiết 5 / Phòng 2	Tiết 3 / Phòng 22
S06	C10	P4	T4–T6	Tiết 5 / Phòng 7	Tiết 2 / Phòng 18
S07	C1	P4	T2–T4	Tiết 6 / Phòng 24	Tiết 3 / Phòng 19
S08	C2	P4	T2–T5	Tiết 5 / Phòng 20	Tiết 3 / Phòng 6
S09	C7	P6	T2–T6	Tiết 5 / Phòng 8	Tiết 6 / Phòng 2
S10	C11	P7	T3–T5	Tiết 3 / Phòng 26	Tiết 1 / Phòng 28
S11	C6	P7	T3–T6	Tiết 1 / Phòng 28	Tiết 4 / Phòng 0
S12	C11	P6	T3–T5	Tiết 4 / Phòng 27	Tiết 1 / Phòng 18
S13	C10	P3	T3–T5	Tiết 3 / Phòng 20	Tiết 3 / Phòng 16
S14	C3	P10	T3–T6	Tiết 5 / Phòng 18	Tiết 6 / Phòng 18
S15	C5	P5	T3–T5	Tiết 6 / Phòng 16	Tiết 2 / Phòng 24
S16	C14	P1	T3–T6	Tiết 4 / Phòng 13	Tiết 2 / Phòng 2
S17	C14	P1	T2–T5	Tiết 1 / Phòng 23	Tiết 5 / Phòng 23
S18	C7	P5	T2–T6	Tiết 2 / Phòng 19	Tiết 5 / Phòng 28
S19	C4	P8	T3–T6	Tiết 3 / Phòng 19	Tiết 6 / Phòng 22
S20	C4	P9	T3–T6	Tiết 2 / Phòng 8	Tiết 5 / Phòng 18
S21	C7	P9	T2–T6	Tiết 1 / Phòng 18	Tiết 3 / Phòng 16
S22	C3	P5	T2–T4	Tiết 1 / Phòng 5	Tiết 1 / Phòng 9
S23	C4	P10	T3–T6	Tiết 1 / Phòng 27	Tiết 2 / Phòng 23
S24	C12	P8	T3–T6	Tiết 6 / Phòng 2	Tiết 2 / Phòng 27

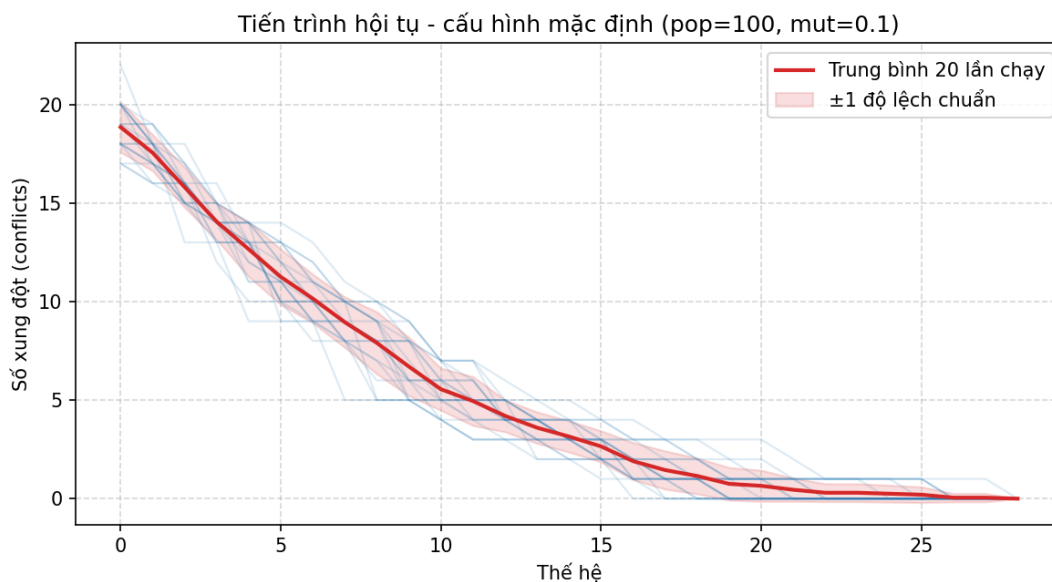
8.2.2 Thống kê qua 20 lần chạy độc lập

Bảng 4: Thống kê tổng hợp 20 lần chạy với cấu hình mặc định (pop = 100, mut = 0.10).

Chỉ số	Giá trị
Số lần chạy thành công (đạt 0 xung đột)	20 / 20 (100%)
Fitness cuối – trung bình $\pm$ độ lệch chuẩn	1.0000 $\pm$ 0.0000
Số xung đột cuối – trung bình $\pm$ độ lệch chuẩn	0.00 $\pm$ 0.00
Thế hệ hội tụ – trung bình $\pm$ độ lệch chuẩn	21.15 $\pm$ 3.48
Thế hệ hội tụ – min / max	16 / 28
Thời gian chạy trung bình (giây)	0.152 $\pm$ 0.039

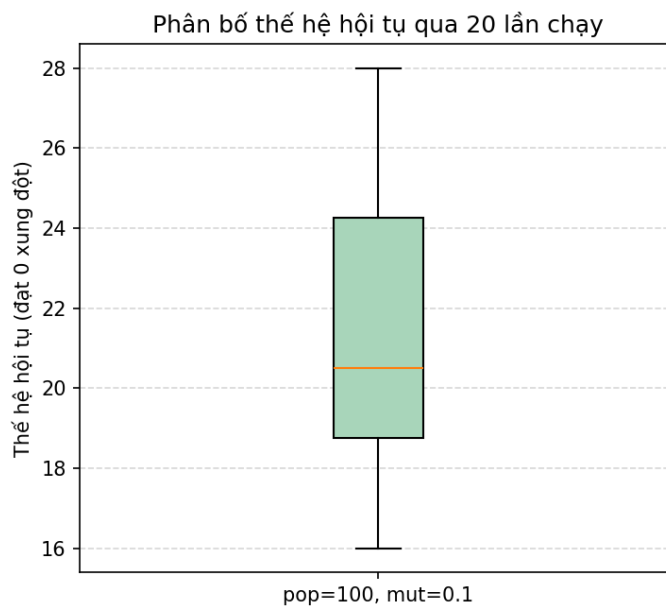
8.2.3 Biểu đồ tiến trình hội tụ

Hình 1 mô tả số xung đột của cá thể tốt nhất theo thế hệ qua 20 lần chạy. Mỗi đường mảnh xanh nhạt là một lần chạy độc lập, đường đồ đậm là trung bình, dải hồng là khoảng ± 1 độ lệch chuẩn. Có thể thấy GA giảm rất nhanh trong khoảng **5–15 thế hệ đầu** — toàn bộ quá trình hội tụ chỉ tốn 16–28 thế hệ — cho thấy bài toán có không gian nghiệm tốt và các phép toán di truyền hoạt động hiệu quả. Việc dải $\pm \sigma$ thu hẹp dần về cuối thể hiện 20 / 20 lần chạy đều đạt cùng chất lượng nghiệm tối ưu.



Hình 1: Tiến trình hội tụ trên 20 lần chạy độc lập với cấu hình mặc định (pop = 100, mut = 0.10).

Hình 2 trình bày phân bố thế hệ hội tụ dưới dạng boxplot, cho thấy độ ổn định của thuật toán giữa các lần chạy: 50% số lần chạy hội tụ trong khoảng ≈ 18 –25 thế hệ.



Hình 2: Phân bố thế hệ hội tụ qua 20 lần chạy.

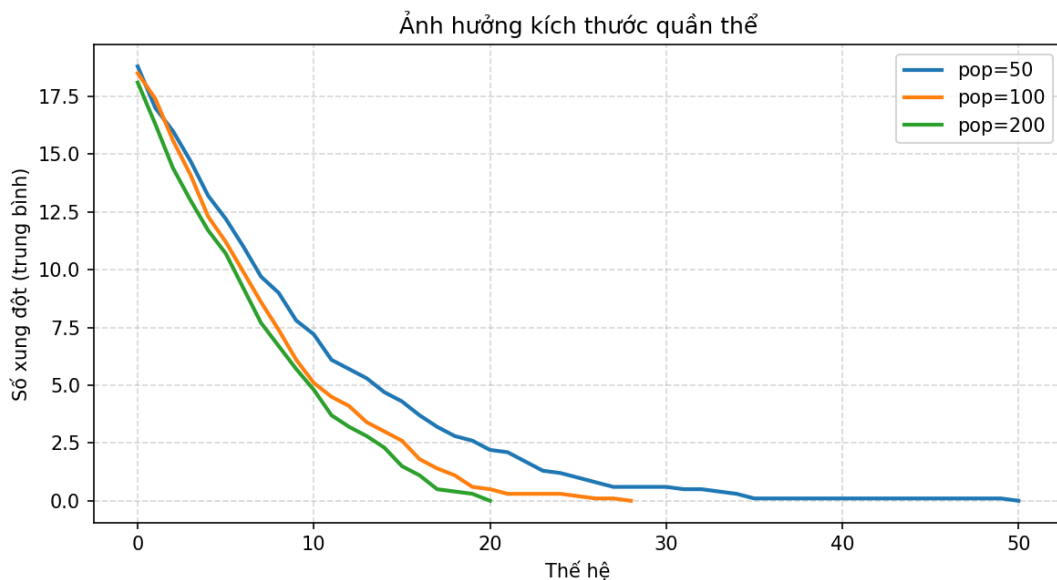
8.3 Khảo sát ảnh hưởng của tham số

8.3.1 Ảnh hưởng kích thước quần thể

Cố định mutation rate = 0.10, thay đổi kích thước quần thể trong {50, 100, 200}. Mỗi cấu hình chạy 10 lần seed độc lập. Bảng 5 và Hình 3 tổng hợp kết quả.

Bảng 5: Ảnh hưởng kích thước quần thể ($\text{mut} = 0.10$, 10 lần / cấu hình).

Pop size	Tỷ lệ thành công	Gen hội tụ TB	Gen hội tụ Std	Thời gian TB (s)
50	10/10	32.1	7.2	0.111
100	10/10	20.9	3.9	0.149
200	10/10	17.8	1.8	0.271



Hình 3: Đường hội tụ trung bình theo kích thước quần thể.

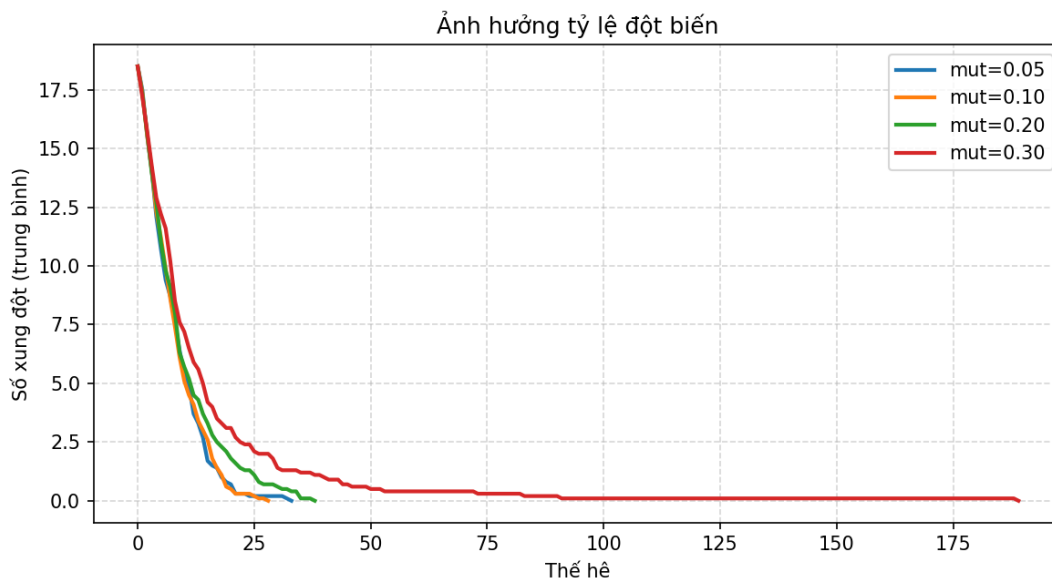
Trên bài toán này cả 3 cấu hình đều đạt 100% thành công, sự khác biệt nằm ở **tốc độ và độ ổn định**. Quần thể nhỏ (50) cần trung bình 32.1 ± 7.2 thế hệ; quần thể lớn (200) hội tụ nhanh nhất với 17.8 ± 1.8 thế hệ nhờ đa dạng di truyền cao. Tuy nhiên chi phí mỗi thế hệ tăng tuyến tính theo `pop_size`, nên *tổng thời gian* của `pop=200` (≈ 0.27 s) cao hơn đáng kể so với `pop=100` (≈ 0.15 s). Cấu hình **100** là lựa chọn cân bằng tốt nhất giữa chất lượng nghiệm và chi phí tính toán.

8.3.2 Ảnh hưởng tỷ lệ đột biến

Cố định `pop = 100`, thay đổi mutation rate trong $\{0.05, 0.10, 0.20, 0.30\}$. Bảng 6 và Hình 4 trình bày kết quả.

Bảng 6: Ảnh hưởng tỷ lệ đột biến (`pop = 100`, 10 lần / cấu hình).

Mutation	Tỷ lệ thành công	Gen hội tụ TB	Gen hội tụ Std	Thời gian TB (s)
0.05	10/10	22.4	5.4	0.164
0.10	10/10	20.9	3.9	0.138
0.20	10/10	30.0	5.0	0.203
0.30	10/10	68.3	45.1	0.451



Hình 4: Đường hội tụ trung bình theo tỷ lệ đột biến.

Trên bài toán này, mọi tỷ lệ trong khảo sát đều đạt 100% thành công nhưng *tốc độ* và *độ ổn định* thay đổi rõ rệt. Mức 0.05–0.10 cho thời gian hội tụ nhanh nhất (≈ 21 thế hệ) và độ lệch chuẩn nhỏ (3.9–5.4). Mức 0.20 chậm hơn (30.0 thế hệ) do đột biến mạnh phá vỡ một số cá thể tốt vừa được lai ghép. Mức 0.30 cho kết quả **bất ổn nhất**: trung bình 68.3 thế hệ với độ lệch chuẩn lên tới 45.1 (lần chạy lâu nhất tới 189 thế hệ) — đột biến quá mạnh khiến quá trình tìm kiếm gần với ngẫu nhiên. Mức **0.10** là điểm *sweet-spot*, khớp với khuyến nghị kinh điển trong tài liệu GA (mutation rate điển hình ≤ 0.2).

8.4 Đánh giá hiệu năng

8.4.1 Tốc độ hội tụ

Cấu hình mặc định đạt 0 xung đột trung bình sau **21.1 thế hệ** (min 16, max 28). Phần lớn mức giảm số xung đột xảy ra trong **5–10 thế hệ đầu** — giai đoạn *khai phá* (exploration) — sau đó GA tinh chỉnh các xung đột còn lại trong khoảng 10–20 thế hệ cuối — giai đoạn *khai thác* (exploitation). Quan sát này khớp với lý thuyết Building-Block của Holland: các khối gene tốt được tổ hợp nhanh ở giai đoạn đầu, sau đó việc tinh chỉnh từng gene cuối cùng cần thêm vài chục thế hệ.

8.4.2 Chất lượng nghiệm

20/20 lần chạy đạt **fitness** = 1.0 (0 xung đột). Lịch học tìm được thoả mãn đồng thời các ràng buộc cứng:

- Hai buổi không liên ngày: được đảm bảo cứng bằng cách mã hoá qua tập $\text{DAY_PAIRS} = \{(M, W), (M, R), (M, F), (T, R), (F, T)\}$, mọi cá thể luôn hợp lệ ràng buộc này.
- Phòng đủ kích thước: kiểm tra `rooms[r].size  $\geq$  class.size`.
- Không trùng phòng giữa các lớp ở cùng (ngày, tiết).
- Không trùng giáo viên ở cùng (ngày, tiết).

8.4.3 Tính ổn định

Độ lệch chuẩn của số xung đột cuối là 0.00, cho thấy thuật toán có **tính ổn định cao** trên bài toán này — mọi lần chạy đều cho cùng chất lượng nghiệm tối ưu. Sự dao động chủ yếu nằm ở *thế hệ hội tụ* ($\sigma \approx 3.5$ với cấu hình mặc định), do điểm xuất phát quần thể ngẫu nhiên khác nhau giữa các seed. So sánh với cấu hình `mut = 0.30` ($\sigma \approx 45.1$), cấu hình mặc định ổn định hơn rõ rệt — củng cố kết luận rằng mutation rate 0.10 là lựa chọn cân bằng tốt nhất giữa tốc độ và tính ổn định.

8.4.4 Thời gian thực thi

Trên cấu hình kiểm thử, mỗi lần chạy mất trung bình ≈ 0.15 giây. Đây là chi phí rất nhỏ cho bài toán quy mô 25 lớp; chi phí scale *gần tuyến tính* theo `population_size` $\times$ số thế hệ thực tế. Việc dừng sớm khi đạt 0 xung đột (*early stopping*) giúp tiết kiệm đáng kể: nhiều lần chạy chỉ cần ~ 20 thế hệ trong tổng ngân sách 500.

8.5 Kết luận mục 6

Thuật toán di truyền với mã hoá chromosome dạng (*cấp ngày, tiết-phòng buổi 1, tiết-phòng buổi 2*) cùng các phép toán **lai ghép đồng nhất theo gene**, **đột biến đa điểm** và **chọn lọc giải đấu** đã giải bài toán xếp lịch lớp với **tỷ lệ thành công 100%** (20 / 20 lần chạy độc lập), **thời gian hội tụ rất nhanh** (trung bình 21.1 thế hệ, ≈ 0.15 giây / lần chạy) và **độ ổn định cao** ($\sigma_{\text{conflicts}} = 0$). Kết quả khảo sát tham số khẳng định cấu hình mặc định (*pop* = 100, *mutation* = 0.10) là lựa chọn cân bằng tốt giữa chất lượng nghiệm, độ ổn định và chi phí tính toán. Bộ kết quả này cho thấy GA là một hướng tiếp cận hiệu quả cho bài toán xếp lịch lớp khi không gian nghiệm lớn và các ràng buộc đa dạng — phù hợp với kỳ vọng của đề bài Topic 3.

9 Tổng kết: Phân tích Thách thức và Hạn chế

9.1 Thách thức

Độ phức tạp của không gian nghiệm.

Bài toán lập lịch lớp học là một bài toán tối ưu tổ hợp (combinatorial optimization) với không gian nghiệm rất lớn. Với số lượng lớp, ngày học, khung giờ và phòng học, số lượng phương án sắp xếp tăng theo cấp số mũ. Điều này gây khó khăn cho việc hội tụ của thuật toán và đảm bảo tìm được nghiệm tối ưu toàn cục.

Xử lý nhiều ràng buộc đồng thời.

Bài toán bao gồm nhiều loại ràng buộc:

- Ràng buộc cứng (hard constraints): trùng phòng, trùng giảng viên, không phù hợp kích thước phòng.
- Ràng buộc cấu trúc: mỗi môn có hai buổi mỗi tuần, không cùng ngày hoặc liền kề.

Việc tích hợp tất cả các ràng buộc này vào hàm fitness là một thách thức lớn, do cần cân bằng giữa các loại vi phạm và tránh thiên lệch trong đánh giá.

Thiết kế biểu diễn chromosome.

Cách biểu diễn nghiệm ảnh hưởng trực tiếp đến hiệu quả của thuật toán. Trong bài này, mỗi lớp được mã hóa thành một vector gồm nhiều thành phần (ngày, slot, phòng), tuy nhiên vẫn có thể sinh ra nhiều nghiệm vi phạm nếu không kiểm soát tốt.

Cân bằng giữa Exploration và Exploitation.

Genetic Algorithm cần cân bằng giữa:

- Exploration: khám phá không gian nghiệm
- Exploitation: khai thác nghiệm tốt hiện tại

Việc lựa chọn mutation rate, crossover strategy và selection method ảnh hưởng lớn đến tốc độ hội tụ và chất lượng nghiệm.

Tính ngẫu nhiên của thuật toán.

Do GA là thuật toán ngẫu nhiên, kết quả có thể khác nhau giữa các lần chạy. Điều này gây khó khăn trong việc đánh giá tính ổn định và tái lập kết quả.

9.2 Hạn chế của mô hình

Chưa bao phủ đầy đủ ràng buộc.

Một số ràng buộc chưa được xử lý hoàn toàn:

- Giới hạn số môn của giảng viên chỉ xử lý khi khởi tạo
- Chưa xét đầy đủ tính hợp lệ của section
- Chưa xét availability của phòng

Hàm fitness còn đơn giản.

Hiện tại:

$$fitness = \frac{1}{1 + conflicts}$$

Hàm fitness này chưa phân biệt mức độ nghiêm trọng của các loại vi phạm và chưa ưu tiên các ràng buộc cứng.

Dữ liệu đầu vào ngẫu nhiên.

Dữ liệu được sinh ngẫu nhiên nên chưa phản ánh đúng tính chất thực tế của bài toán.

Hiệu suất tính toán.

Khi số lượng lớp hoặc thể hệ tăng, thời gian chạy tăng đáng kể. Chưa có các kỹ thuật tối ưu như song song hóa hoặc caching.

Thiếu cơ chế xử lý ràng buộc nâng cao.

Mô hình hiện tại chỉ sử dụng penalty để xử lý vi phạm, chưa có:

- Repair function
- Constraint-preserving operators

10 Kết luận và Hướng phát triển

10.1 Kết luận

Trong bài toán này, Genetic Algorithm đã được áp dụng để giải quyết bài toán lập lịch lớp học với nhiều ràng buộc phức tạp. Kết quả cho thấy GA có khả năng:

- Tìm kiếm nghiệm khả thi trong không gian lớn
- Giảm dần số lượng xung đột qua các thế hệ

Tuy nhiên, nghiệm tìm được chưa đảm bảo tối ưu toàn cục và hiệu quả phụ thuộc nhiều vào thiết kế hàm fitness và cách mã hóa bài toán.

10.2 Hướng phát triển

Cải tiến hàm fitness.

Phân loại ràng buộc thành:

- Hard constraints
- Soft constraints

Sử dụng hàm fitness có trọng số:

$$fitness = \frac{1}{1 + \alpha \cdot hard + \beta \cdot soft}$$

Cải tiến xử lý ràng buộc.

Áp dụng:

- Repair function
- Constraint-preserving operators

Cải tiến toán tử di truyền.

- Crossover có cấu trúc
- Mutation có định hướng

Thuật toán lai (Hybrid).

Kết hợp GA với:

- Simulated Annealing

- Local Search (Hill Climbing)

Tối ưu hiệu suất.

- Parallel Genetic Algorithm
- GPU acceleration

Sử dụng dữ liệu thực tế.

Áp dụng vào các hệ thống thực:

- Lịch học đại học
- Phòng học chuyên dụng
- Lịch rảnh của giảng viên

Xây dựng hệ thống trực quan.

Phát triển:

- Giao diện hiển thị timetable
- Dashboard phân tích conflict

Tài liệu

- [1] TS. Nguyễn Quốc Minh, *Homework 3 - Nhập môn Trí Tuệ Nhân Tạo (CO3061)*, Trường Đại học Bách Khoa - ĐHQG HCM, 2026.
- [2] Colorni, A., Dorigo, M., & Maniezzo, V. (1990). Genetic algorithms and highly constrained problems: The time-table case. In *Parallel Problem Solving from Nature*. Springer, Berlin, Heidelberg.
- [3] Burke, E. K., & Newall, J. P. (2004). Solving examination timetabling problems through adaption of heuristic orderings. *Annals of Operations Research*.
- [4] Abdullah, S., Burke, E. K., & McCollum, B. (2007). A hybrid evolutionary approach to the university course timetabling problem. In *2007 IEEE Congress on Evolutionary Computation*. IEEE.
- [5] Sutar, S. R., & Bichkar, R. S. (2012). University timetabling based on hard constraints using genetic algorithm. *International Journal of Computer Applications*
- [6] Holland, J. H. (1975). *Adaptation in natural and artificial systems*. University of Michigan Press.
- [7] Goldberg, D. E. (1989). *Genetic algorithms in search, optimization and machine learning*. Addison-Wesley.
- [8] De Jong, K. A. (1975). *An analysis of the behavior of a class of genetic adaptive systems*. Doctoral dissertation, University of Michigan.
- [9] Bickel, T., & Thiele, L. (1995). A mathematical analysis of tournament selection. In *Proceedings of the 6th International Conference on Genetic Algorithms* (pp. 9–16).
- [10] Syswerda, G. (1989). Uniform crossover in genetic algorithms. In *Proceedings of the 3rd International Conference on Genetic Algorithms* (pp. 2–9).
- [11] Mühlenbein, H. (1992). How genetic algorithms really work: Mutation and hillclimbing. In *Parallel Problem Solving from Nature* (pp. 15–26).
- [12] Bäck, T. (1993). *Evolutionary algorithms in theory and practice*. Oxford University Press.
- [13] Srinivas, M., & Patnaik, L. M. (1994). Adaptive probabilities of crossover and mutation in genetic algorithms. *IEEE Transactions on Systems, Man, and Cybernetics*, 24(4), 656–667.
- [14] Eiben, A. E., & Smith, J. E. (2015). *Introduction to evolutionary computing* (2nd ed.). Springer.
- [15] Lewis, R. (2008). A survey of metaheuristic-based techniques for university timetabling problems. *OR Spectrum*, 30(1), 167–190.